Table des matières

Cours 12 — Rebâtir le pipeline de bout en bout 1
Ce que vous allez faire 1
Avant de commencer 2
Étape 1 — Mettre de côté, puis effacer · De bout en bout 2
Étape 2 — Le pipeline sur un projet vide · De bout en bout 3
Étape 3 — Les deux chemins, en une commande · De bout en bout 3
Étape 4 — Prouver · De bout en bout 5
Étape 5 — Passer le projet · De bout en bout 8
À retenir 9

Cours 12 — Rebâtir le pipeline de bout en bout

Cours 12/12 · La Brûlerie — Formation Data Engineering

Sources	**Ingestion**	**RAW**
d'où vient la donnée	aller la chercher, et la charger	la déposer, la ranger, la traiter
✓ ici · De bout en bout, les deux chemins, montés en entier	✓	✓

Ce que vous allez faire

Onze cours ont bâti La Brûlerie morceau par morceau, sur un entrepôt et un lac qu'on n'a jamais effacés. Un projet est fini quand il se rebâtit de zéro en une commande, qu'une preuve dit qu'il est revenu le même, et qu'une autre personne peut le reprendre sans vous. Ce cours n'apprend presque rien de neuf : il met le projet à ces trois épreuves. Il efface tout ce que Data Engineering a fait et qui n'est pas une source, trouve les gestes que vous aviez faits à la main, les range dans une commande qui rebâtit les deux chemins, le froid et le chaud, prouve le résultat, puis étiquette la version livrée.

mettre de côté, puis effacer → le pipeline sur un projet vide → `reconstruire.cmd`, les deux chemins en une commande → l'argent, dans le pipeline → prouver : les règles, l'entrepôt d'avant, la caisse → passer le projet

Les fichiers à écrire se donnent en pseudocode, et le code complet est dans `Codes_12_DE.md`, à ouvrir après avoir essayé.

© 2026 iAlexMG. Tous droits réservés.

Avant de commencer

Deux fenêtres de l'Anaconda Prompt servent à ce cours, toutes deux placées dans `Projet_Brulerie` : la première pour le Bloc-notes, Python et Git ; la seconde pour le notebook, qui s'ouvre à l'étape 4. Dans la première :

```
conda activate formations
REM Attendu : (formations) au début de l'invite.
git -C 01_de status
REM Attendu : nothing to commit, working tree clean.
python 01_de\pipeline.py 2025-06-30
REM Attendu : bronze : 1527 tickets rangés, 1527 à la source, puis raw : 75 fichiers au journal,
REM puis controles : 4 règles jugées, avertir : un seul export par jour (1).
```

Ce cours part du projet tel que le cours 11 l'a laissé, commité. Un fichier modifié et pas commité ferait mentir la reconstruction, qui doit partir de ce que Git garde. S'il en reste un, reprenez la fin du cours 11.

Étape 1 — Mettre de côté, puis effacer · De bout en bout

« De zéro » ne veut pas dire « de rien ». Les sources restent : ce sont les fichiers du café, et le journal du direct, écrit par le rejoueur du cours 09, en est une aussi. Le code reste, puisque Git le garde. Tout le reste se refait, et part :

Ce qui vit dans `Projet_Brulerie`	**Ce que c'est**	**Son sort**
`sources`, `plateforme\direct\journal.jsonl`	ce que le café et son torréfacteur livrent	reste
`01_de`	le code, dans Git	reste
`plateforme\entrepot\brulerie.duckdb`	la `RAW`, l'argent des lots, le direct, et ce qu'Analytics Engineering y a bâti, si vous l'avez fait	mis de côté ; puis ses seules tables de Data Engineering effacées
`plateforme\direct\decalage.txt`	où le consommateur en est dans le journal	effacé avec la `RAW`, sinon le consommateur croirait avoir tout lu
`plateforme\lac`	le bronze, la météo, la table Iceberg et son catalogue	effacé

L'entrepôt est partagé : Data Engineering n'y efface que ses tables, par `vider_tables_de.py`, comme aux cours 06 et 08, et ce qu'Analytics Engineering y a bâti reste. La première preuve comparera l'entrepôt rebâti à celui d'avant : on en garde d'abord une copie, dans `plateforme\avant`. La météo et la table Iceberg partent avec le lac. Ce sont des essais des cours 04 et 10, hors des deux chemins : leur script les refait, `python 01_de\meteo.py` avec Internet, et `python 01_de\iceberg.py`.

1.1 Mettre de côté, puis effacer, dans la première fenêtre.

© 2026 iAlexMG. Tous droits réservés.

```
mkdir plateforme\avant
copy plateforme\entrepot\brulerie.duckdb plateforme\avant\brulerie.duckdb
REM Attendu : 1 fichier(s) copié(s).
python ..\01_Data_Engineering\01_Brulerie_facile\Paquet_de_depart\vider_tables_de.py
REM Attendu : Tables de Data Engineering effacées : 11 (argent_lots, direct_courbes, journal, puis les
REM huit raw_…), et Objets gardés, ceux d'Analytics Engineering : 0, ou davantage si vous l'avez fait.
del plateforme\direct\decalage.txt
rmdir /s /q plateforme\lac
dir /b plateforme
REM Attendu : avant, direct et entrepot.
```

Étape 2 — Le pipeline sur un projet vide · De bout en bout

Le pipeline tourne chaque nuit depuis le cours 05. Lancez-le sur le projet vide, comme si c'était sa première nuit.

2.1 Lancer le pipeline.

```
python 01_de\pipeline.py 2025-06-30
REM Attendu : bronze : échec, IO Error: Failed to create directory "plateforme/lac/bronze/tickets":
REM Le chemin d'accès spécifié est introuvable.
echo %ERRORLEVEL%
REM Attendu : 1.
```

Le premier geste fait à la main se montre. `COPY … PARTITION_BY` crée le dernier dossier de son chemin, `tickets`, mais pas ses parents : au cours 02, vous aviez créé `plateforme\lac\bronze` vous-même, une fois, et plus personne n'y a pensé. Le pipeline ne l'a jamais vu manquer, parce que le lac n'a jamais été effacé. Le geste ne sert qu'à partir de zéro : il va dans la commande qui rebâtit.

Étape 3 — Les deux chemins, en une commande · De bout en bout

Une reconstruction ne demande aucune commande à la main au milieu : tout ce qu'elle fait tient dans un fichier du dépôt, `reconstruire.cmd`, qui monte les deux chemins l'un après l'autre. Le chemin froid, c'est le pipeline, du bronze aux règles ; le chemin chaud, c'est le consommateur du cours 09, qui relit le journal du direct depuis le début, puisque son décalage est parti.

Chaque commande du fichier se termine par `|| exit /b 1`. `||` lance ce qui le suit quand la commande a échoué, et `exit /b 1` arrête le fichier avec le code 1 : la reconstruction s'arrête au premier échec et le dit par son code de sortie, comme le pipeline.

Pour remplir	L'outil
un dossier créé seulement s'il manque	`if not exist <dossier> mkdir <dossier>` ; un projet neuf, comme celui de la personne qui reprendra le dépôt à l'étape 5, n'a pas même

© 2026 iAlexMG. Tous droits réservés.

Pour remplir	L'outil
	`plateforme\entrepot`, et DuckDB ne le crée pas : la ligne qui suit le trou le crée
le chemin chaud	le consommateur du cours 09, avec son seuil de 218 °C, et son `exit /b 1` en cas d'échec
les accents	aucun dans le fichier : l'Anaconda Prompt lit un `.cmd` dans une autre page de codes que le Bloc-notes, et `é` y deviendrait un autre caractère

3.1 Créer le fichier.

```
notepad 01_de\reconstruire.cmd
```

Accepter la création, puis écrire le fichier à partir de ce pseudocode, les trous ____ compris :

```
@echo off
rem 01_de/reconstruire.cmd : rebatir les deux chemins de zero, en une commande, depuis Projet_Brulerie
rem 1. les dossiers du bronze et de l'entrepot : ni COPY ni DuckDB ne creent ceux du dessus
if not exist plateforme\lac\bronze ____
if not exist plateforme\entrepot mkdir plateforme\entrepot
rem 2. le chemin froid : le bronze, la RAW et l'argent jusqu'au 30 juin, juges par leurs regles
python 01_de\pipeline.py 2025-06-30 || exit /b 1
rem 3. le chemin chaud : le journal du direct, relu depuis son decalage, l'alerte a 218 degres

____
echo Reconstruction : au vert.
```

Enregistrer, puis fermer le Bloc-notes.

3.2 Rebâtir.

```
01_de\reconstruire.cmd
REM Attendu : bronze : 1527 tickets rangés, 1527 à la source, raw : 75 fichiers au journal, puis
REM controles : échec, Catalog Error: Table with name argent_lots does not exist!, suivi de
REM Did you mean "raw_lots"? et de la ligne 15 de controles.sql.
echo %ERRORLEVEL%
REM Attendu : 1.
```

Le second geste fait à la main se montre : l'argent des lots, que le cours 08 refaisait par `duckdb plateforme\entrepot\brulerie.duckdb -f`, à côté du pipeline. La règle 4 le juge, et il n'existe plus. Ce geste-là ne sert pas qu'à partir de zéro : depuis le cours 08, chaque nuit chargeait de nouvelles courbes dans la `RAW` sans refaire l'argent, qui prenait du retard en silence. Il va donc dans le pipeline, juste après la `RAW`, dans la même connexion à l'entrepôt.

3.3 Ouvrir le pipeline.

© 2026 iAlexMG. Tous droits réservés.

```
notepad 01_de\pipeline.py
```

Remplacer la fonction `raw` par celle-ci, le trou compris :

```
def raw(jusqua):
    # 2. la RAW, chargée jusqu'au jour donné ; puis l'argent, refait en entier à partir d'elle
    with duckdb.connect("plateforme/entrepot/brulerie.duckdb") as con:
        charger(con, jusqua)
        ____
        return f"{con.execute('SELECT COUNT(*) FROM journal').fetchone()[0]} fichiers au journal"
```

Enregistrer, puis fermer le Bloc-notes. Le fichier compte 30 lignes, son plafond.

3.4 Rebâtir encore. La `RAW` chargée au premier essai ne double pas : le journal la connaît.

```
01_de\reconstruire.cmd
REM Attendu : bronze, raw et controles comme avant le cours, puis
REM ALERTE 2025-06-10 08:56:00 : lot L-2025-0229, grain à 218.4 °C,
REM 762 messages lus depuis le décalage 0, 725 lectures nouvelles, et Reconstruction : au vert.
echo %ERRORLEVEL%
REM Attendu : 0.
```

Les deux chemins sont revenus, en une dizaine de secondes. Relancée, la commande ne refait que ce qui se refait en entier, le bronze et l'argent, et dit `0 messages lus depuis le décalage 762`.

3.5 Valider la commande et le pipeline.

```
git -C 01_de add pipeline.py reconstruire.cmd
git -C 01_de commit -m "Cours 12 : l'argent dans le pipeline, et la commande qui rebâtit"
REM Attendu : 2 files changed.
```

Étape 4 — Prouver · De bout en bout

`Reconstruction : au vert` dit que chaque commande a rendu 0, et le cours 11 a montré ce que vaut un vert. Refaire n'est pas prouver. Trois preuves s'empilent, chacune voyant ce que les autres ne voient pas :

La preuve	Ce qu'elle compare	Ce qu'elle ne voit pas
les règles	chaque passage contre ce qui devait arriver, au cours 11	ce qu'aucune règle ne demande
l'entrepôt d'avant	chaque table rebâtie contre celle des onze cours, ligne pour ligne, dans les deux sens	ce qui était déjà faux avant

© 2026 iAlexMG. Tous droits réservés.

La preuve	Ce qu'elle compare	Ce qu'elle ne voit pas
la caisse	le bronze rebâti contre la caisse lue à la source, sans passer par le pipeline	ce que la caisse elle-même aurait mal compté

La première est passée à l'étape 3 : quatre règles jugées, aucune à bloquer.

4.1 Ouvrir le notebook. Seconde Anaconda Prompt : activer `formations`, se placer dans `Projet_Brulerie`, puis lancer DuckDB.

```
duckdb -ui
```

Créer un notebook et le renommer « Cours 12 ». Dans sa première cellule, attacher les deux entrepôts en lecture seule, chacun sous son nom :

```
ATTACH IF NOT EXISTS 'plateforme/entrepot/brulerie.duckdb' AS refait (READ_ONLY);
ATTACH IF NOT EXISTS 'plateforme/avant/brulerie.duckdb' AS avant (READ_ONLY);
-- Attendu : refait et avant paraissent sous Attached databases.
```

4.2 Comparer les cinq tables que Data Engineering lit, dans une cellule neuve. C'est la comparaison du cours 06, étendue à l'argent et au direct.

```
-- 1. chaque table, dans les deux sens : ce qui manque au refait, puis ce qu'il a en trop
SELECT 'raw_tickets' AS table_refaite,
       (SELECT COUNT(*) FROM (SELECT * FROM avant.raw_tickets EXCEPT ALL SELECT * FROM refait.raw_tickets)) AS manquent,
       (SELECT COUNT(*) FROM (SELECT * FROM refait.raw_tickets EXCEPT ALL SELECT * FROM avant.raw_tickets)) AS en_trop
UNION ALL
SELECT 'raw_lots',
       (SELECT COUNT(*) FROM (SELECT * FROM avant.raw_lots EXCEPT ALL SELECT * FROM refait.raw_lots)),
       (SELECT COUNT(*) FROM (SELECT * FROM refait.raw_lots EXCEPT ALL SELECT * FROM avant.raw_lots))
UNION ALL
SELECT 'raw_courbes',
       (SELECT COUNT(*) FROM (SELECT * FROM avant.raw_courbes EXCEPT ALL SELECT * FROM refait.raw_courbes)),
       (SELECT COUNT(*) FROM (SELECT * FROM refait.raw_courbes EXCEPT ALL SELECT * FROM avant.raw_courbes))
UNION ALL
SELECT 'argent_lots',
       (SELECT COUNT(*) FROM (SELECT * FROM avant.argent_lots EXCEPT ALL SELECT * FROM refait.argent_lots)),
       (SELECT COUNT(*) FROM (SELECT * FROM refait.argent_lots EXCEPT ALL SELECT * FROM avant.argent_lots))
UNION ALL
SELECT 'direct_courbes',
       (SELECT COUNT(*) FROM (SELECT * FROM avant.direct_courbes EXCEPT ALL SELECT * FROM refait.direct_courbes)),
       (SELECT COUNT(*) FROM (SELECT * FROM refait.direct_courbes EXCEPT ALL SELECT * FROM avant.direct_courbes));
-- Attendu : 5 lignes, 0 et 0 pour chacune.
```

4.3 Comparer le journal, par le nom et l'empreinte de chaque fichier ; l'instant du chargement diffère forcément.

© 2026 iAlexMG. Tous droits réservés.

```
SELECT (SELECT COUNT(*) FROM refait.journal) AS fichiers,
       (SELECT COUNT(*) FROM (SELECT fichier, empreinte FROM avant.journal
                              EXCEPT ALL SELECT fichier, empreinte FROM refait.journal)) AS manquent,
       (SELECT COUNT(*) FROM (SELECT fichier, empreinte FROM refait.journal
                              EXCEPT ALL SELECT fichier, empreinte FROM avant.journal)) AS en_trop;
-- Attendu : 75 fichiers, 0 et 0.
```

L'entrepôt rebâti égale celui des onze cours : 1 585 lignes de tickets, 47 lots, 3 718 lectures, 47 lots d'argent et 725 lectures du direct, et rien de plus ; le journal répond des cinq autres tables de la `RAW`, chargées des mêmes fichiers, aux mêmes empreintes. Cette preuve ne voit pas ce qui était déjà faux avant : un défaut des onze cours serait revenu tel quel. D'où la troisième.

4.4 Retomber sur la caisse. La caisse se lit ici à la source, sans le code du pipeline : tout en texte, le dernier export de chaque jour gardé, les montants convertis en `DECIMAL`. Puis le bronze rebâti, lu dans le lac.

```
-- 1. la caisse lue à la source, le dernier export de chaque jour, puis le bronze rebâti
WITH mois AS (
    SELECT 'caisse' AS lu_dans, type_ticket, statut, CAST(sous_total AS DECIMAL(10, 2)) AS sous_total,
           CAST(encaisse AS DECIMAL(10, 2)) AS encaisse
    FROM read_csv('sources/caisse/*/tickets_*.csv', all_varchar = true, filename = true)
    QUALIFY filename = MAX(filename) OVER (PARTITION BY split_part(filename, '_', 2))
    UNION ALL
    SELECT 'bronze', type_ticket, statut, sous_total, encaisse
    FROM read_parquet('plateforme/lac/bronze/tickets/*/*.parquet')
)
-- 2. ce que le café a vendu et encaissé ; un ticket annulé n'encaisse rien
SELECT lu_dans, COUNT(*) AS tickets,
       COUNT(*) FILTER (WHERE type_ticket = 'VENTE' AND statut = 'PAYE') AS ventes_payees,
       SUM(sous_total) FILTER (WHERE statut = 'PAYE') AS avant_taxes, SUM(encaisse) AS encaisse
FROM mois
GROUP BY lu_dans
ORDER BY lu_dans;
-- Attendu : 2 lignes pareilles, bronze et caisse : 1527 tickets, 1511 ventes payées, 16718.00 avant
-- taxes, 18666.38 encaissés.
```

Juin retombe sur la caisse, au cent : 1 511 ventes payées, 16 718,00 $ avant taxes, le remboursement de 4,00 $ déduit, et 18 666,38 $ encaissés. Les quinze tickets annulés comptent parmi les 1 527, sans un sou.

4.5 Fermer le notebook : fermer l'onglet du navigateur, puis quitter DuckDB dans la seconde Anaconda Prompt.

```
.quit
```

Attendu : l'invite revient, et la seconde fenêtre peut se fermer.

© 2026 iAlexMG. Tous droits réservés.

Étape 5 — Passer le projet · De bout en bout

Un projet se passe par écrit. Chaque fichier dit, en première ligne, ce qu'il fait ; il manque ce que lit la personne qui reprendra le dépôt : par où commencer. C'est le `README.md`, à la racine du dépôt, que GitHub et les éditeurs affichent en premier. Il ne raconte pas le code : il dit quoi avoir avant de commencer, quoi lancer, d'où, et ce qui doit sortir. Son épreuve : quelqu'un qui ne vous connaît pas rebâtit le projet en le suivant, sans vous appeler.

5.1 Écrire le `README.md`, dans la première fenêtre.

```
notepad 01_de\README.md
```

Un fichier Markdown : `#` ouvre un titre, `-` une puce, et l'accent grave entoure un nom de fichier ou une commande. `<!-- … -->` y est un commentaire, que l'affichage cache. Accepter la création, puis écrire chaque section dans vos mots :

```
# 01_de — les données de La Brûlerie du Passage

<!-- 1. ce qu'est le dépôt, et ce qu'il livre -->
(trois ou quatre phrases : le café, ses deux sources, ce que le dépôt en tire, du bronze au direct,
et le chiffre qui prouve qu'il dit vrai)

## Ce qu'il faut avoir

<!-- 2. ce qui n'est pas dans le dépôt : les sources, le journal du direct, les outils -->
- `Projet_Brulerie/sources/`, les fichiers du café, intacts ;
(puis le journal du direct, et l'environnement conda, avec les versions des outils)

## Rebâtir

<!-- 3. la commande, d'où la lancer, et ce qui doit sortir -->
(ce qu'on efface d'abord, la commande de l'étape 3, ce qu'elle affiche)

## Faire tourner et vérifier

<!-- 4. une nuit, un rattrapage, les règles, le test -->
(chaque commande, avec son code de sortie)
```

Enregistrer, puis fermer le Bloc-notes.

5.2 Valider le `README.md`, puis étiqueter la version. Une étiquette annotée, `tag -a`, porte un auteur, une date et un message, comme un commit. `v1.0` nomme ce commit, et le nommera encore quand le dépôt aura avancé.

```
git -C 01_de add README.md
git -C 01_de commit -m "Cours 12 : le README du dépôt"
REM Attendu : 1 file changed.
```


© 2026 iAlexMG. Tous droits réservés.

```
git -C 01_de tag -a v1.0 -m "La Brûlerie, les deux chemins rebâtis de zéro et prouvés"
git -C 01_de log --oneline --decorate -1
REM Attendu : (HEAD -> master, tag: v1.0) Cours 12 : le README du dépôt ; main à la place de master
REM selon votre Git.
git -C 01_de status
REM Attendu : nothing to commit, working tree clean.
```

Le dépôt `01_de`, étiqueté `v1.0`, est la pièce que vous pouvez montrer.

5.3 Faire le ménage. L'entrepôt d'avant a fini son travail de témoin.

```
rmdir /s /q plateforme\avant
dir /b plateforme
REM Attendu : direct, entrepot et lac.
```

À retenir

- Un projet est fini quand il se rebâtit de zéro en une commande : tout ce qui n'est pas une source se refait, et la commande vit dans le dépôt.
- Une reconstruction trouve les gestes faits à la main : un dossier créé une fois, une table refaite à côté du pipeline. Ce qui ne sert que de zéro va dans la commande, ce qui sert chaque nuit va dans le pipeline.
- Refaire n'est pas prouver : les règles, l'entrepôt d'avant ligne pour ligne, puis la caisse lue à la source, au cent.

© 2026 iAlexMG. Tous droits réservés.